

0484 Bases de dades

RA5: Programació



1. Introducció

La majoria dels SGBD incorporen la possibilitat d'ampliar la funcionalitat amb la programació

Per exemple Oracle incorpora PL/SQL

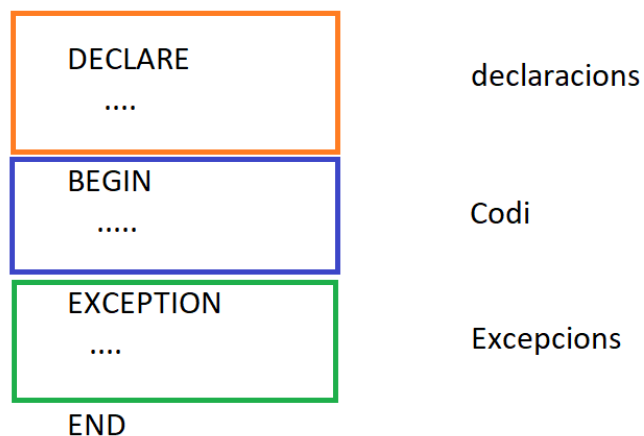
2. Introducció a PL/SQL

La programació bàsica és per unes estructures anomenades blocs. Un bloc PL/SQL té 3 parts:

Zona de declaracions

Zona de codi

Zona de excepcions



3. Print

Per mostrar informació per pantalla farem servir la instrucció

DBMS_OUTPUT.PUT_LINE()

Des de terminal la sortida per pantalla no està activada per defecte de forma que a cada sessió caldrà activar-la amb `set serveroutput on;`

```
SQL> set serveroutput on;
```

```
begin
    dbms_output.put_line('Hola k ase?');
end;
/
```

```
SQL> set serveroutput on;
SQL> begin
    2  dbms_output.put_line('Hola k ase?');
    3  end;
    4  /
Hola k ase?

PL/SQL procedure successfully completed.
```

4. Sintaxi:

- Totes les instruccions acaben amb ;
- No es case sensitive
- Per assignar un valor a una variable farem servir :=
- operadors:

+ , - , * , /
|| → Per concatenar

5. Variables:

Les variables van a la zona de declaracions

NOM TIPUS [:= valor inicial];

```
declare
    x int;
    nom varchar2(20);
    y float;
    z date;
    w boolean;
    dni char(9);
begin
    x:=3;
    nom := 'Jordi';
    dbms_output.put_line(nom);
end;
/
```

```
DECLARE
    x varchar2(20);
BEGIN
    x:='Jordi';
    DBMS_OUTPUT.put_line('Saludos: ' || x);
END;
/
```

6. Conditionals:

La sintaxi és:

a) if condicio then

end if;

b) if condicio then

else

end if;

```
declare
  x int;
  nom varchar2(20);
begin
  x:=3;
  nom := 'Jordi';
  dbms_output.put_line('Hola:' || nom);
  if x>=18 THEN
    dbms_output.put_line('Ets major d"edat');
  else
    dbms_output.put_line('Ets menor d"edat');
  end if;
end;
/
```

c) IF condicio THEN

...

elsif

...

END IF;

7. Bucles

Disposem de:

- a) WHILE condicio LOOP

...
End loop;

Per exemple:

```
declare
    x integer;
BEGIN
    x:=1;
    while x<=10 LOOP
        DBMS_OUTPUT.put_line(x);
        x:=x+1;
    end loop;
end;
/
```

- b) FOR variable IN Min..MAX LOOP

end loop;
Per exemple:

```
BEGIN
    FOR x IN 1..10 LOOP
        DBMS_OUTPUT.put_line(x);
    end loop;
end;
/
```

Observa que no cal declarar la variable del for, no cal donar-li valor inicial ni cal incrementar a cada iteració

Exercici: Crear la taula de multiplicar d'un número

```
declare
    numero int:=3;
    i int;
```

```
BEGIN
    for i in 0..10 loop
        DBMS_OUTPUT.put_line(numero || ' x ' || i || ' = ' || numero*i);
    end loop;
end;
/
```

Exercici: Mostra per pantalla els números de l'1 al 10 // mostrar un triangle de 10 files, a la primera fila el valor 1, a la segona fila 12, a la tercera 123...

```
declare
    i integer;
begin
    i:=10;
    while i>=0 LOOP
        dbms_output.put_line(i);
        i:=i-1;
    end loop;
end;
/

begin
    for i in reverse 0..10 LOOP
        dbms_output.put_line(i);
    end loop;
end;
/
```

```
declare
    texto varchar2(30):="";
begin
    for i in 1..10 loop
        texto := texto || i || ' ';
        dbms_output.put_line(texto);
    end loop;
end;
/
```

Exercici: Taula de multiplicar del 3

```
declare
    taula integer:=3;
    resultat integer;
begin
    for i in 0..10 loop
        resultat:=i*taula;
        dbms_output.put_line(taula || ' * ' || i || ' = ' || resultat);
    end loop;
end;
/
```

8. Procediments

Fins ara els blocs que hem vist són anònims això implica que no es guarda a la base de dades

Els procediments són blocs de codi però als que podem posar un nom. La sintaxi és:

CREATE [OR REPLACE] PROCEDURE nom IS

Ara ja no cal la paraula DECLARE

Per executar un procediment farem

execute nom

Per exemple:

```
create procedure problema5 is
    taula integer:=3;
    resultat integer;
begin
    for i in 0..10 loop
        resultat:=i*taula;
        dbms_output.put_line(taula || ' * ' || i || ' = ' || resultat);
    end loop;
end;
/
```

Els procediments també poden rebre paràmetres

CREATE [OR REPLACE] PROCEDURE nom (param1 tipus, param2 tipus) IS

```
create or replace procedure problema5 (tabla integer, limite integer) is
    resultat integer;
begin
    for i in 0..limite loop
        resultat:=i*tabla;
        dbms_output.put_line(tabla || ' * ' || i || ' = ' || resultat);
    end loop;
end;
/
```

9. Funcions

Igual que els procediments podem declarar funcions. Les funcions són similars però retornen un valor

CREATE OR REPLACE FUNCTION NOM (parametres) RETURN tipus IS

Per exemple:

```
create or replace function suma100(valor integer) return integer IS
BEGIN
    return valor+100;
end;
/
```

Per poder provar funcions haurem de fer consultes. A Oracle disposem de la taula dual per poder fer proves:

```
SQL> select suma100(20) from dual;

SUMA100(20)
-----
        120
```

```
/* Problema 9 */
```

```
create or replace procedure problema9(entrada varchar2) is
    x integer;
BEGIN
    x:=length(entrada);
    dbms_output.put_line('Longitud: ' || x);
    dbms_output.put_line(upper(entrada));
    dbms_output.put_line(lower(entrada));
end;
```

```
/

BEGIN
problema9('hola');
end;

/* problema 10 */
create or replace procedure problema10(entrada varchar2)
is
    x integer;
    aux varchar2(1);
BEGIN
    -- x:=instr(entrada, ' ');
    x:=0;
    for i in 1..length(entrada) LOOP
        aux:=substr(entrada,i,1);
        if aux=' ' and x=0 THEN
            x:=i;
        end if;
    end loop;

    if (x=0) THEN
        dbms_output.put_line('No hi ha cap espai');
    ELSE
        dbms_output.put_line('Espai en blanc a: ' || x);
    end if;
end;
/
```

```
/* problema 11 */
create or replace procedure problema11(entrada varchar2)
is
    aux varchar2(1);
    sortida varchar2(2500):="";
BEGIN
    for i in REVERSE 1..length(entrada) loop
        aux:=substr(entrada, i, 1);
        sortida:=sortida || aux;
    end loop;
    dbms_output.put_line(sortida);
end;
/
```

Exercici: Converteix el procediment anterior a funció

```
create or replace function problema11_fun(entrada varchar2)
return varchar2 is
    aux varchar2(1);
    sortida varchar2(2500):="";
BEGIN
    for i in REVERSE 1..length(entrada) loop
        aux:=substr(entrada, i, 1);
        sortida:=sortida || aux;
    end loop;
```

```
        return sortida;
    end;
/

/* per poder executar funcions necessitem un bloc
per recuperar el valor que ens retorna */
DECLARE
x varchar2(100);
begin
    x:=problema11_fun('Hola');
    dbms_output.put_line(x);
end;
/

/* o podem fer servir la taula dual que és per fer proves */
select problema11_fun('Hola') from dual;
```

10.- Accés a bases de dades

Des de PL/SQL podem accedir a la base de dades per fer petites operacions SQL de forma nativa.

Per treballar podem crear una taula per fer proves:

```
CREATE table persona(
codi integer primary key,
nom varchar2(100) not null,
sou float check(sou>1000)
);
```

10.1.- Inserts

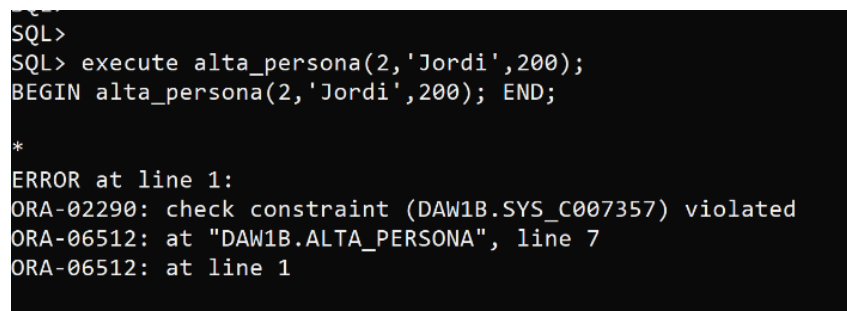
Per donar d'alta una nova persona farem servir l'insert com sempre.

Com a paràmetres tindrem codi, nom i sou

```
create or replace procedure alta_persona (  
                                p_codi integer,  
                                p_nom varchar2,  
                                p_sou FLOAT  
    ) is  
begin  
    insert into persona values(p_codi, p_nom, p_sou);  
end;  
/
```

Ja funciona, però què succeeix si algú intenta executar:

```
execute alta_persona(1,'Jordi', 200);
```



```
SQL>  
SQL> execute alta_persona(2,'Jordi',200);  
BEGIN alta_persona(2,'Jordi',200); END;  
  
*  
ERROR at line 1:  
ORA-02290: check constraint (DAW1B.SYS_C007357) violated  
ORA-06512: at "DAW1B.ALTA_PERSONA", line 7  
ORA-06512: at line 1
```

Es produeix un error donat que hi ha una restricció per validar que el sou ha de ser superior a 1000

Ens hauriem de defensar per evitar que passi aixó. Podem modificar el codi i afegir un condicional

```
create or replace procedure alta_persona (  
    p_codi integer,  
    p_nom varchar2,  
    p_sou FLOAT  
) is  
begin  
  
    if p_sou >1000 then  
        insert into persona values(p_codi, p_nom, p_sou);  
    ELSE  
        dbms_output.put_line('Error: El sou ha de ser superior  
a 1000');  
    end if;  
  
end;
```

```
SQL> execute alta_persona(2,'Jordi',200);  
Error: El sou ha de ser superior a 1000  
  
PL/SQL procedure successfully completed.
```

També hauriem de validar que el nom és obligatori:

```
SQL> execute alta_persona(2,NULL,2000);  
BEGIN alta_persona(2,NULL,2000); END;  
  
*  
ERROR at line 1:  
ORA-01400: cannot insert NULL into ("DAW1B"."PERSONA"."NOM")  
ORA-06512: at "DAW1B.ALTA_PERSONA", line 9  
ORA-06512: at line 1
```

```
create or replace procedure alta_persona (  
                                p_codi integer,  
                                p_nom varchar2,  
                                p_sou FLOAT  
) is  
begin  
    if p_nom is NOT NULL then  
        if p_sou >1000 then  
            insert into persona values(p_codi, p_nom,  
p_sou);  
        ELSE  
            dbms_output.put_line('Error: El sou ha de ser  
superior a 1000');  
        end if;  
    ELSE  
        dbms_output.put_line('Error: El nom és obligatori');  
    end if;  
end;
```

Finalment ens quedarà per validar que el codi és correcte

```
SQL> execute alta_persona(2,'Jordi',2000);  
BEGIN alta_persona(2,'Jordi',2000); END;  
  
*  
ERROR at line 1:  
ORA-00001: unique constraint (DAW1B.SYS_C007358) violated  
ORA-06512: at "DAW1B.ALTA_PERSONA", line 9  
ORA-06512: at line 1
```

10.2.- Consultes simples

Una consulta simple és la que retorna 1 fila!!!

Per treballar tenim la consulta:

```
SELECT
INTO
FROM
WHERE
```

...

Per exemple, per mostrar la informació d'un empleat:

```
create or replace procedure cons_persona IS
    x varchar2(20);
    y float;
BEGIN
    select nom, SOU
    into x,y
    from empleats
    where codi_Empl=8;
    dbms_output.put_line('*****');
    dbms_output.put_line('***** INFO PERSONA *****');
    dbms_output.put_line('*****');
    dbms_output.put_line('Nom: ' || x);
    dbms_output.put_line('Sou: ' || y);

end;
/
```

Ampliació: Podriem ampliar el codi per tal de passar el codi de la persona a mostrar?

```
create or replace procedure cons_empl (p_codi integer) IS
    v_codi integer
    v_nom varchar2(20);
    v_sou float;
BEGIN
    select codi, nom, SOU
    into v_codi,v_nom, v_sou
    from persona
    where codi=p_codi;
    dbms_output.put_line('*****');
    dbms_output.put_line('***** INFO PERSONA *****');
    dbms_output.put_line('*****');
    dbms_output.put_line('Codi: ' || v_codi);
    dbms_output.put_line('Nom: ' || v_nom);
    dbms_output.put_line('Sou: ' || v_sou);

end;
/
```

Però el select into ha de retornar una fila, altrament ens donarà error.
Podem defensar-nos mirant primer si existeix aquest codi a la base de dades

```
create or replace procedure cons_empl (p_codi integer) IS
    v_codi integer
    v_nom varchar2(20);
    v_sou float;
    aux integer;
BEGIN

    select count(*)
    into aux
    from persona where codi=p_codi;

    select codi, nom, SOU
    into v_codi,v_nom, v_sou
    from persona
    where codi=p_codi;
    dbms_output.put_line('*****');
    dbms_output.put_line('***** INFO PERSONA *****');
    dbms_output.put_line('*****');
    dbms_output.put_line('Codi: ' || v_codi);
    dbms_output.put_line('Nom: ' || v_nom);
    dbms_output.put_line('Sou: ' || v_sou);
end;
/
```

Podem fer una funció encarregada de consultar si existeix l'empleat

```
create or replace function existeix_persona (p_codi integer) return integer is
    aux integer;
BEGIN

    select count(*)
    into aux
    from persona where codi=p_codi;

    return aux;
end;
/

create or replace procedure cons_empl (p_codi integer) IS
```

```
v_codi integer
v_nom varchar2(20);
v_sou float;
aux integer;
BEGIN

    aux:=existeix_persona(p_codi);

    if aux <>0 then

        select codi, nom, SOU
        into v_codi,v_nom, v_sou
        from persona
        where codi=p_codi;
        dbms_output.put_line('*****');
        dbms_output.put_line('***** INFO PERSONA *****');
        dbms_output.put_line('*****');
        dbms_output.put_line('Codi: ' || v_codi);
        dbms_output.put_line('Nom: ' || v_nom);
        dbms_output.put_line('Sou: ' || v_sou);
    end;
/
```

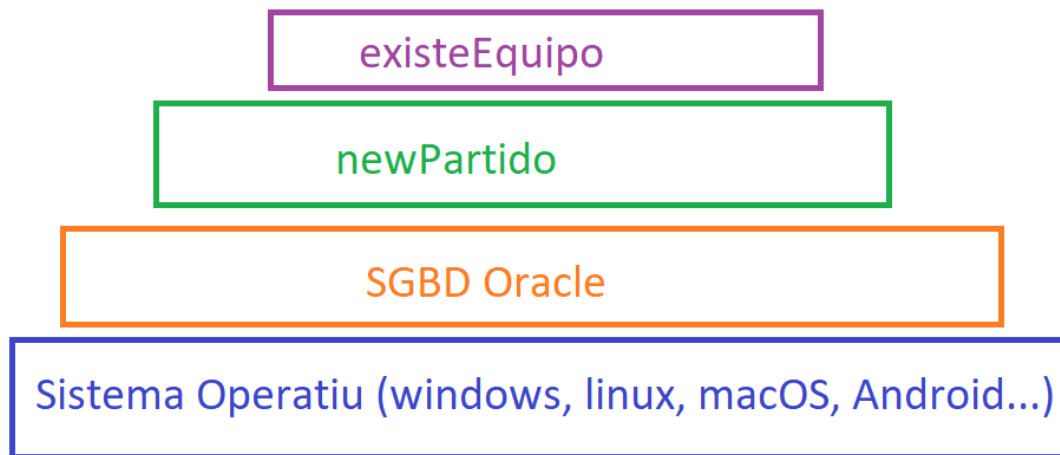
Exercici: Acabar l'insert per verificar que el codi no existeix abans de donar d'alta.

Exercici: Fer el delete i el update

11. Excepcions

Una excepció és un error en temps d'execució, per exemple una divisió per 0, error de memòria, disc dur....

. Per poder executar una funció o procediment, en realitat es crea una pila



Quan un codi produeix un error, aquest error s'ha de tractar. Altrament, la funció acaba inesperadament i passa el control al següent nivell. Si aquest nivell el sap tractar bé, altrament, acaba i passa el control al següent nivell...

Per exemple, el següent codi pot provocar error

```
create or replace procedure divideix(x integer, y integer)
IS
    resultat float;
BEGIN
    resultat:=x/y;
    dbms_output.put_line('El resultat és: ' || resultat);
end;
/
```

```
SQL> execute divideix(2,0)
BEGIN divideix(2,0); END;

*
ERROR at line 1:
ORA-01476: divisor is equal to zero
ORA-06512: at "DAM1A23.DIVIDEIX", line 5
ORA-06512: at line 1

SQL>
```

Per defensar-nos podem treballar amb excepcions.

A oracle tenim algunes excepcions ja predefinides entre les que destaquen:

Zero_divide → es produeix quan es fa una divisió per 0

Too_many_rows

No_data_found → Quan el select..into no retorna res

Value_error → Es produeix quan fem una operació de tipus incorrecte

DUP_VAL_ON_INDEX → Es produeix quan intentem introduir la clau primària i ja existeix

Others

Per exemple:

```
create or replace procedure divideix(x integer, y integer)
IS
    resultat float;
BEGIN
    resultat:=x/y;
    dbms_output.put_line('El resultat és: ' || resultat);
EXCEPTION
    when zero_divide then
        dbms_output.put_line('No pots dividir per 0');

end;
/
```

12. Registres

Podem millorar les consultes amb els registres

Es fan declarant una variable de tipus ROWTYPE

```
create or replace procedure cons_empl (p_codi integer) IS
    personita persona%Rowtype;
BEGIN
    select *
    into personita
    from persona
    where codi_Empl=p_codi;
    dbms_output.put_line('*****');
    dbms_output.put_line('***** INFO PERSONA *****');
    dbms_output.put_line('*****');
    dbms_output.put_line('Nom: ' || personita.nom);
    dbms_output.put_line('Sou: ' || personita.sou);

end;
/
```

13. Cursors

les consultes que hem fet fins ara només poden retornar 1 resultat.

Per tractar consultes més complexes tenim els cursors. Per gestionar-los cal:

- a) Declarar
 - b) Obrir
 - c) Tractar
 - d) Tancar
-
- a) Per declarar:
CURSOR nom IS consultaSQL;
 - b) Per obrir
OPEN nom;
 - c) Per tractar: Cal fer un bucle
Els cursors ens permeten treballar amb una sèrie de propietats i mètodes:
 - FETCH: Retorna 1 resultat de la consultat. Va de resultat en resultat.
 - Quan es fa el fetch, a més, s'activen algunes variables:
 - NOTFOUND: Booleà que es posa a true (cert) quan el fetch no retorna res
 - FOUND
 - NUMROWS: número de fetchs que s'han realitzat
 - d) Per tancar
CLOSE nom;

```
create or replace procedure llista_perones is
    cursor c_persona IS
        select *
        from persona
        order by sou;
    x c_persona%ROWTYPE;
BEGIN
    open c_persona;
    dbms_output.put_line('NOM          SOU');
    LOOP
        fetch c_persona into x;
        exit when c_persona%NOTFOUND;
        dbms_output.put_line(x.nom || chr(9) || x.sou);
    END LOOP;
```

```
        close c_empl;  
end;  
/
```

amb el bucle for ens podem estalviar algunes instruccions

```
create or replace procedure llista_persones is  
    cursor c_persona IS  
        select *  
        from persona  
        order by sou;  
BEGIN  
    dbms_output.put_line('NOM   COGNOM   SOU');  
    for x in c_persona LOOP  
        dbms_output.put_line(x.nom || chr(9) || x.sou);  
    END LOOP;  
end;  
/
```